Crescita delle funzioni

Riccardo Di Michele

Crescita del costo computazionale

Abbiamo già visto come calcolare **il tempo di esecuzione** di un algoritmo, per esempio, l'algoritmo *insertion sort* ha come costo computazionale:

$$f(n) = an^2 + bn + c$$

Ma spesso avremmo bisogno di informazioni in più. Vogliamo sapere quanto questo tempo **cresce** all'aumentare dell'input.
Prendendo come esempio il costo analizzato prima proviamo a fare un calcolo matematico: proviamo a vedere come si comporta quel costo per *n molto grandi*, proviamo quindi a calcolare il suo **limite**.

$$\lim_{n\to\infty} an^2 + bn + c \longrightarrow \lim_{n\to\infty} n^2 + n \longrightarrow \lim_{n\to\infty} n^2$$

Poichè siamo di fronte a un limite, valori come i coefficienti diventano insignificanti per $n \to \infty$ e il termine n^2 invece "domina" sui termini di grado inferiore. Diremo quindi che l'algoritmo di *insertion sort*, per un input sempre più grande, crescerà *come* n^2, oppure, in maniera più corretta, diremmo $\Theta(n^2)$, *"theta grande di* n^2*"*.
Introduciamo allora il concetto della **notazione asintotica**, ovvero definire i *limiti* di crescita di un certo costo per input sempre più grandi. I vari tipi di notazione (vedremo nel dettaglio dopo) servono a dichiarare il ***lower bound*** e l'***upper bound***, che corrispondono al *limite inferiore* e al *limite superiore* della nostra funzione. L'*upper bound* sarà indicato da $O(g(n))$, da leggere come *"O grande di* $g(n)$*"*, mentre il *lower bound* da $\Omega(g(n))$, letto come *"Omega grande di* $g(n)$*"*.
Una nota importante: nonostante esistano diverse notazioni dobbiamo sottolineare il fatto che sono solo e soltanto delle <u>nomenclature</u>: quando leggiamo $\Theta(n)$ oppure $\Omega(n^2)$ è solo un modo per descrivere l'andamento di una funzione al crescere di n, non dobbiamo leggere queste notazioni come "*a* n^2 *applico la proprietà* Ω".

Caso peggiore e caso migliore

Dobbiamo prima però fare una distinzione molto importante tra i ***casi dell'algoritmo***. Prendiamo come esempio il seguente algoritmo per calcolare una potenza *ennesima* di 2.

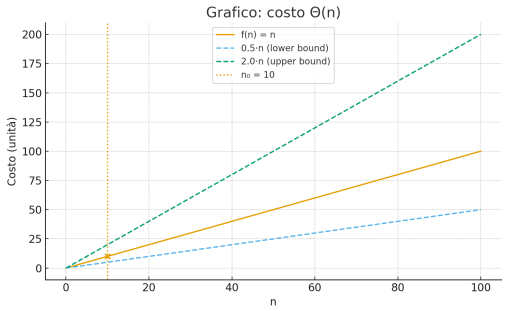
```
function power_2(n)
  f = 1
  for i = 1 to n do
    f = f * 2
  end for

  return f
```

In questo caso il costo dell'algoritmo è **fisso** poichè farà sempre n cicli. Troveremo però algoritmi il cui numero di cicli (e costo) non sarà fisso, per esempio l'algoritmo di ***binary search***.

```
function binary_search(A, n, T)
  L = 0
  R = n - 1
  while L <= R do
    m = L + floor((R - L) / 2)
    if A[m] < T then
      L = m + 1
    else if A[m] > T then
      R = m - 1
    else
      return m
  return unsuccessful
```

È importante notare che, a differenza del primo algoritmo, questo presenta un *if statement* capace di far terminare l'algoritmo in qualsiasi momento. Questo rende impossibile stabilire un numero finito di cicli, quindi, dovremmo studiare l'algoritmo a seconda del ***caso migliore*** (minor numero di cicli) e ***caso peggiore*** (maggior numero di cicli).
Facendo un esempio con l'algoritmo sopra citato il *caso migliore* sarebbe quando $A[m]$ coincide con T al primo ciclo, avendo un costo computazionale pari a $f(n) = 1$; il *caso peggiore* sarebbe invece quando l'algoritmo non trova l'elemento T, e avremmo così un costo di $f(n) = log_2(n) + 1$. Una volta che abbiamo chiaro questi concetti possiamo continuare spiegando i tipi di *notazioni asintotiche*.



Notazione asintotica Θ

Formalmente diremmo che

$$\Theta(g(n)) = f(n) \iff \exists\ c_1, c_2, > 0,\ \ n_0 > 0 \mid c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)\ \forall n \geq n_0$$

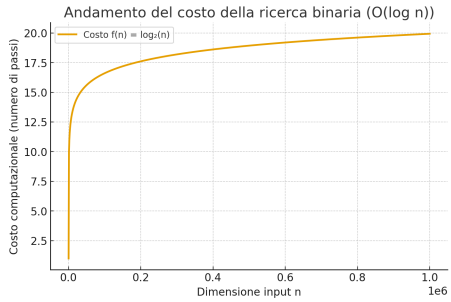
dopo un certo indice n_0 il nostro *costo computazionale* $f(n)$ sarà compreso dentro una curva che avrà $c_1 \cdot g(n)$ come limite superiore e $c_2 \cdot g(n)$ come limite inferiore. Poichè c_1 e c_2 possono essere qualsiasi valore positivo troveremo più di una funzione del tipo $c \cdot g(n)$ che soddisfa la proprietà enunciata, diremo infatti che $\Theta(g(n))$ è un'insieme di queste funzioni; proprio per questo motivo sarebbe più corretto scrivere $f(n) \in \Theta(g(n))$ ma la notazione $f(n) = \Theta(g(n))$ ha lo stesso valore.

L'algoritmo preso in figura è il **power_2** analizzato nel paragrafo prima e possiamo notare, come detto poc'anzi, che in realtà esistono altri possibili lower bound e upper bound; in parole povere potevamo anche prendere un "intervallo" più piccolo.

Notazione asintotica O

Abbiamo già parlato di come la *notazione* Θ limita una funzione sia superiormente che inferiormente, ma spesso ci sarà possibile indicare solo il limite superiore, in quel caso useremo la notazione O.

$$O(g(n)) = f(n) \iff \exists\ c > 0, n_0 > 0 \mid 0 \leq f(n) \leq c \cdot g(n)\ \forall\ n \geq n_0$$



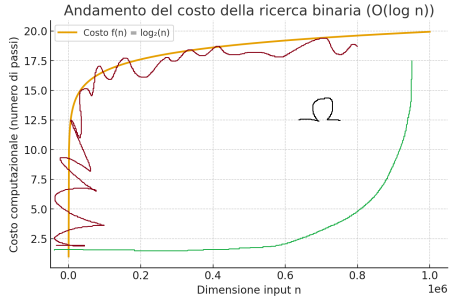
per tutti i valori a "*destra*" di n_0 i valori di $f(n)$ coincidono o si trovano sopra $c \cdot g(n)$.
Ovviamente una funzione che appartiene già a un insieme $\Theta(g(n))$ sarà anche appartenente all'insieme $O(g(n))$, poichè una funzione in Θ è munita sia di limite superiore che inferiore; possiamo quindi dire che l'insieme $\Theta \subseteq O$.

L'algoritmo in figura questa volta è il *binary search* un algoritmo dove non possiamo calcolare un numero fisso di cicli poichè al suo interno vi sono presenti degli *if statement* che possono far terminare l'algoritmo, di conseguenza ci limiteremo a calcolare il suo costo nel caso peggiore, ovvero $log_2(n)$.
Diremo allora che $f(n) = O(log_2(n))$.

Notazione asintotica Ω

Ovviamente, la notazione Ω indicherà invece un limite inferiore di una certa funzione; formalmente diremo

$$\Omega(g(n)) = f(n) \iff \exists\ c > 0, n_0 > 0 \mid 0 \leq c \cdot g(n) \leq f(n)\ \forall\ n \geq n_0$$



per tutti i valori a "*destra*" di n_0 i valori di $f(n)$ coincidono o si trovano sopra $c \cdot g(n)$.

Questo è l'algoritmo del pisello duro e mi serve questo paragrafo solo come placeholder ciao haha lol

Proprietà principale delle notazioni asintotiche